

Labwork 6: Map

Philippe Olivier Schriqui-Duque
ID: 2640042

Introduction

In this labwork I implemented three map operations on images: binarization, brightness control and blending. In a map, the same function is applied to every pixel without looking at the neighbours, so each GPU thread handles one pixel. I reused the 2D kernel structure of Labwork 4 and the same 4K image (3840×2880).

For each part, I ran the GPU function 20 times per block size (copies included) and computed the mean time and the standard deviation. The first run is not timed because of the compilation.

Part A: Binarization

Each gray pixel becomes 0 or 1 depending on a threshold τ (default 128, can be given with `sys.argv`):

$$b(x, y) = \begin{cases} 0 & \text{if } \Phi(x, y) < \tau \\ 1 & \text{if } \Phi(x, y) \geq \tau \end{cases}$$

I first make the image gray with my grayscale kernel from Labwork 4, then I run the binarization kernel. The gray image stays on the GPU between the two kernels. I save 255 instead of 1 so the white is visible.

```
@cuda.jit
def binarize(src, dst, threshold):
    x = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    y = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if x < src.shape[1] and y < src.shape[0]:
        value = 0 if src[y, x, 0] < threshold else 255
        dst[y, x, 0] = value
        dst[y, x, 1] = value
        dst[y, x, 2] = value
```

8x8	14.40 +/- 1.70 ms
16x8	14.43 +/- 2.17 ms
16x16	13.48 +/- 1.71 ms
32x8	13.45 +/- 1.28 ms
16x32	13.47 +/- 1.23 ms
32x16	13.51 +/- 1.79 ms
32x32	13.74 +/- 1.31 ms

All block sizes are very close, the differences are smaller than the standard deviation.

Part B: Brightness control

I add the same value to the three channels of every pixel (default 50, positive to make it brighter, negative to make it darker). The pixels are `uint8`, so I convert them to `int` first, otherwise $250 + 10$ gives 4. Then I clamp the result between 0 and 255.

```
for c in range(3):
    new = int(src[y, x, c]) + value
    if new > 255: new = 255
    if new < 0: new = 0
    dst[y, x, c] = new
```

8x8	14.11	+/-	1.52	ms
16x8	12.54	+/-	1.35	ms
16x16	11.97	+/-	1.46	ms
32x8	12.33	+/-	1.57	ms
16x32	12.50	+/-	1.33	ms
32x16	13.04	+/-	2.16	ms
32x32	12.36	+/-	1.03	ms

16x16 is the fastest and 8x8 the slowest, but most sizes stay within the noise. 8x8 blocks only have 64 threads, so the limit on blocks per SM keeps the GPU from being fully used.

Part C: Blending two images

Two images of the same size are combined with a coefficient c (default 0.5):

$$Q(x, y) = c \times \Phi_1(x, y) + (1 - c) \times \Phi_2(x, y)$$

I cropped and resized the second photo to 3840×2880 so both images have the same size. Since c is between 0 and 1, the result stays between 0 and 255.

```
for ch in range(3):
    dst[y, x, ch] = c * src1[y, x, ch] + (1 - c) * src2[y, x, ch]
```

8x8	18.18	+/-	2.81	ms
16x8	18.89	+/-	2.37	ms
16x16	16.78	+/-	1.31	ms
32x8	17.16	+/-	1.71	ms
16x32	19.13	+/-	1.96	ms
32x16	23.08	+/-	5.13	ms
32x32	22.57	+/-	2.00	ms

The blending is slower than the two other parts because two images are copied to the GPU. 16x16 is again the fastest. The 32x16 result has a big standard deviation (5.13 ms), so it is probably a few slow runs and not a real effect.

Conclusion

The map pattern is easy to put on the GPU: one thread per pixel and no communication between threads, only the calculation line changes from one part to another. These operations are so simple that the copies between the CPU and the GPU take most of the time, so the block size has very little impact.